

Module 3

Topics: Sub-setting of SAS datasets, Data cleaning, Creating new variables

Files to download: phase1.sas7bdat (from Module 2), phase2.sas7bdat (from Module 2), labs.sas7bdat (from Module 2), mod3lab.sas7bdat

3.1 - Merging of Partial SAS Datasets

In the last lab we merged data files that contained different variables for the same observations. The data sets to be merged had to have a variable in common that served as a unique identifier for each observation.

Sometimes you may want to merge data files in which all of the observations in one dataset are not contained in the other. For example, you may be interested only in patients that were enrolled in Phase 2 of your study. In this case, you would like to merge the lab data contained in LABS with the PHASE2 dataset. Let's see what happens when we merge these two datasets using the following program:

First, we must sort the datasets on the common variable:

```
libname NICOLE 'H:\AEPI537D';

PROC SORT DATA = NICOLE.PHASE2;
  BY STUDYID;
RUN;

PROC SORT DATA = NICOLE.LABS;
  BY STUDYID;
RUN;
```

Now we will merge these two datasets:

```
DATA P2WLABS;
  MERGE NICOLE.PHASE2 NICOLE.LABS;
  BY STUDYID;
RUN;
```

Now let's take a look at the resulting dataset:

```
PROC PRINT DATA= P2WLABS;
RUN;
```

You will notice that the new dataset contains all 30 observations contained in the LABS dataset along with their lab results. However, the data from the phase dataset are missing for 20 of these observations. That is because we used only the PHASE2 dataset in the merge. In order to get a final dataset that only contains the observations from the second phase of the study we must use an IN=

clause during our merge. The IN= allows us to control which observations are included in the final dataset. The following code illustrates the use of an IN= clause:

```
DATA P2WLABS2;  
    MERGE NICOLE.PHASE2(IN=A) NICOLE.LABS;  
    BY STUDYID;  
    IF A;  
RUN;
```

Now let's look at the new dataset:

```
PROC PRINT DATA= P2WLABS2;  
RUN;
```

The (IN=A) part of the SAS statement assigns the value A to every observation contained in the PHASE2 dataset. Then the IF A statement tells SAS that the new dataset should contain only those observations assigned the value A. Our new dataset contains all of the phase 2 variables plus the lab result value from the lab dataset for only those observations in the second phase of the study.

3.2 - More on Merging

Now let's get a little more practice merging with IN statements.

We need to make a few datasets. You can either copy, paste, and run the following blocks of code into SAS to create the 3 datasets we will be working with for this next section OR use the SAS code provided in the SAS program "Mod 3 SAS code for dataset creation".

If you copy and paste from this document, make sure you copy and paste exactly so the spacing is maintained or the datasets will not store appropriately. Look at each dataset to make sure the variables stored correctly. If your DATALINES code doesn't look exactly like mine, I recommend using the SAS program provided.

First, we will make a dataset called WALKDEAD that lists 20 characters from the show The Walking Dead. We will give each character a STUDYID from 1001-1020, a first name (FNAME), a last name (LNAME), a sex (SEX) with values of 1 for male and 2 for female, a variable with the number of episodes the character appears in (NUMEPIS), and a variable with the name of the show (SHOW).

```

DATA WALKDEAD;
INPUT @1 STUDYID @6 FNAME $8. @15 LNAME $10. @25 SEX @27 NUMEPIS @31 SHOW $20.;
DATALINES;
1001 RICK GRIMES 1 115 THE WALKING DEAD
1002 CARL GRIMES 1 115 THE WALKING DEAD
1003 DARYL DIXON 1 113 THE WALKING DEAD
1004 CAROL PELETIER 2 112 THE WALKING DEAD
1005 MAGGIE GREENE 2 108 THE WALKING DEAD
1006 MICHONNE . 2 96 THE WALKING DEAD
1007 GLENN RHEE 1 81 THE WALKING DEAD
1008 TARA CHAMBLER 2 71 THE WALKING DEAD
1009 SASHA WILLIAMS 2 69 THE WALKING DEAD
1010 EUGENE PORTER 1 68 THE WALKING DEAD
1011 ROSITA ESPINOSA 2 68 THE WALKING DEAD
1012 GABRIEL STOKES 1 63 THE WALKING DEAD
1013 AARON . 1 54 THE WALKING DEAD
1014 MORGAN JONES 1 53 THE WALKING DEAD
1015 BETH GREENE 2 51 THE WALKING DEAD
1016 ABRAHAM FORD 1 38 THE WALKING DEAD
1017 HERSHEL GREENE 1 38 THE WALKING DEAD
1018 DWIGHT . 1 36 THE WALKING DEAD
1019 LORI GRIMES 2 36 THE WALKING DEAD
1020 ANDREA HARRISON 2 35 THE WALKING DEAD
;
RUN;

```

Next, we will make a dataset called GOT that lists 20 characters from the show Game of Thrones. We will give each character a STUDYID from 5001-5020, a first name (FNAME), a last name (LNAME), a sex (SEX) with values of 1 for male and 2 for female, a variable with the number of episodes the character appears in (NUMEPIS), and a variable with the name of the show (SHOW).

```

DATA GOT;
INPUT @1 STUDYID @6 FNAME $16. @23 LNAME $10. @33 SEX @35 NUMEPIS @38 SHOW $20.;
DATALINES;
5001 TYRION LANNISTER 1 61 GAME OF THRONES
5002 CERSEI LANNISTER 2 58 GAME OF THRONES
5003 DAENERYS TARGARYEN 2 56 GAME OF THRONES
5004 JON SNOW 1 56 GAME OF THRONES
5005 SANSA STARK 2 54 GAME OF THRONES
5006 ARYA STARK 2 53 GAME OF THRONES
5007 JAIME LANNISTER 1 49 GAME OF THRONES
5008 JORAH MORMONT 1 48 GAME OF THRONES
5009 THEON GREYJOY 1 43 GAME OF THRONES
5010 SAMWELL TARLY 1 43 GAME OF THRONES
5011 PETYR Baelish 1 41 GAME OF THRONES
5012 LORD VARYS 1 41 GAME OF THRONES
5013 BRIENNE OF TARTH . 2 37 GAME OF THRONES
5014 DAVOS SEAWORTH 1 36 GAME OF THRONES
5015 BRAN STARK 1 35 GAME OF THRONES
5016 BRONN . 1 34 GAME OF THRONES
5017 MISSANDEI . 2 34 GAME OF THRONES
5018 SANDOR CLEGANE 1 33 GAME OF THRONES
5019 GRAND MAESTER PYCELLE 1 31 GAME OF THRONES
5020 EDDISON TOLLETT 1 30 GAME OF THRONES
;
RUN;

```

AEPI537D

Finally, we will make a dataset called ACTORS that lists 45 actors that play characters from the shows The Walking Dead (n=20), Game of Thrones (n=20), and Empire (n=5). Each actor will have a STUDYID that corresponds to the STUDYID of the character they play in the other datasets we just made. Note that we did not make a dataset of Empire characters so STUDYID 8001-8005 are sort of like extra records.

```
DATA ACTORS;
INPUT @1 STUDYID @6 ACTOR $25. ;
DATALINES;
1001 ANDREW LINCOLN
1002 CHANDLER RIGGS
1003 NORMAN REEDUS
1004 MELISSA MCBRIDE
1005 LAUREN COHAN
1006 DANAI GURIRA
1007 STEVEN YEUN
1008 ALANNA MASTERSON
1009 SONEQUA MARTIN-GREEN
1010 JOSH MCDERMITT
1011 CHRISTIAN SERRATOS
1012 SETH GILLIAM
1013 ROSS MARQUAND
1014 LENNIE JAMES
1015 EMILY KINNEY
1016 MICHAEL CUDLITZ
1017 SCOTT WILSON
1018 AUSTIN AMELIO
1019 SARAH WAYNE CALLIES
1020 LAURIE HOLDEN
5001 PETER DINKLAGE
5002 LENA HEADEY
5003 EMILIA CLARKE
5004 KIT HARRINGTON
5005 SOPHIE TURNER
5006 MAISIE WILLIAMS
5007 NIKOLAJ COSTER-WALDAU
5008 IAIN GLEN
5009 ALFIE ALLEN
5010 JOHN BRADLEY
5011 AIDAN GILLEN
5012 CONLETH HILL
5013 GWENDOLINE CHRISTIE
5014 LIAM CUNNINGHAM
5015 ISAAC HEMPSTEAD WRIGHT
5016 JEROME FLYNN
5017 NATHALIE EMMANUEL
5018 RORY MCCANN
5019 JULIAN GLOVER
5020 BEN CROMPTON
8001 BRYSHERE GRAY
8002 JUSSIE SMOLLETT
8003 TERRENCE HOWARD
8004 TARAJI P HENSON
8005 GABOUREY SIDIBE
;
RUN;
```

Now that you have SAS datasets WALKDEAD, GOT, and ACTORS, we can try putting these together in different ways.

- A. CONCATENATE the WALKDEAD dataset and the GOT dataset so we have all of our show characters in a single dataset. These 2 datasets have the SAME VARIABLES (studyid, fname, lname, sex, numepis, show) but different people (WALKDEAD has studyid range 1001-1020 and GOT has studyid range 5001-5020) so we will use a SET statement to stack one on top of the other.

```
DATA WD_GOT;
    SET WALKDEAD GOT;
RUN;
```

Open the Viewtable or use PROC PRINT to look at the WD_GOT dataset. The new dataset WD_GOT has all 20 Walking Dead characters from the WALKDEAD dataset and all 20 Game of Thrones characters from the GOT dataset.

- B. MERGE the ACTORS dataset with the WD_GOT so we can see which actor plays each character.

First, we will need to SORT each dataset by the common variable (studyid).

```
PROC SORT DATA = WD_GOT;
    BY STUDYID;
RUN;

PROC SORT DATA = ACTORS;
    BY STUDYID;
RUN;
```

Let's start by merging these together with no special settings and see what it looks like.

```
DATA WDGOT_A;
    MERGE WD_GOT ACTORS;
    BY STUDYID;
RUN;

PROC PRINT DATA= WDGOT_A NOOBS; RUN;
```

When we run a PROC PRINT, we notice that it merged the 2 datasets together, but some of the information is missing for the Empire actors because we didn't have their characters represented in the WD_GOT dataset.

- C. To fix this, MERGE the datasets again, but this time tell SAS to only keep people that are in the WD_GOT character dataset (so we are basically dropping the extra Empire records in the ACTORS dataset. Adding IN=CHAR after the WD_GOT dataset assigns the CHAR nickname to every row in WD_GOT. Adding IF CHAR tells SAS to only keep records in WDGOT_B if they have been assigned a value of CHAR from the IN statement.

```

PROC SORT DATA = WD_GOT;
    BY STUDYID;
RUN;

PROC SORT DATA = ACTORS;
    BY STUDYID;
RUN;

DATA WDGOT_B;
    MERGE WD_GOT (IN=CHAR) ACTORS;
    BY STUDYID;
    IF CHAR;
RUN;

PROC PRINT DATA= WDGOT_B NOOBS; RUN;

```

Our new dataset, WDGOT_B has only 40 observations, just the records that were in the WD_GOT dataset, but it merged in the extra info for each row from the ACTORS dataset.

- D. You can also ask SAS to combine records by including records that are NOT in one of the datasets. Let's do the same merge again, but this time we will ask SAS to keep only the records that are in the ACTORS dataset and are NOT in the WD_GOT dataset. In the merge below, IF ACT means "if the record is present in the ACTORS dataset" and IF ^CHAR means "if the record is not present in the WD_GOT dataset".

```

PROC SORT DATA = WD_GOT;
    BY STUDYID;
RUN;

PROC SORT DATA = ACTORS;
    BY STUDYID;
RUN;

DATA WDGOT_C;
    MERGE WD_GOT (IN=CHAR) ACTORS (IN=ACT);
    BY STUDYID;
    IF ACT AND ^CHAR;
RUN;

PROC PRINT DATA= WDGOT_C NOOBS; RUN;

```

Adding IN=CHAR after the WD_GOT dataset assigns the CHAR nickname to every row in WD_GOT. Adding IN=ACT after the ACTORS dataset assigns the ACT nickname to every row in ACTORS. Adding IF ACT AND ^CHAR tells SAS to only keep records in WDGOT_C if they have been assigned a value of ACT AND have NOT been assigned a value of CHAR, so only people that are in the ACTORS dataset that are not in the WD_GOT dataset.

As you can see, our new dataset, WDGOT_C has only 5 observations, just the people that were in the ACTORS dataset that were NOT in the WD_GOT dataset (the 5 Empire actors). Note that SAS merged the datasets and included all the variables from both, but we have mostly missing

information for these records - Empire actors, STUDYID 8001-8005, because they were not present in the WD_GOT dataset.

- E. You can use IN= statements to do very complicated merging: Give you people from only one of the datasets being combined, people from one and NOT the other, only people that are in both, etc..

IN statements can also be used to assign values to new variables during a merge. For example, let's say I want to merge the full lists together (all the character data and all the actor data) and use IN= statements to give SAS some of the missing information for people that are in the actors dataset but not in the characters dataset (so, the Empire actors). To demonstrate this, let's do exactly the same merge we did earlier to make WDGOT_A, but now we will also assign some missing info for the Empire characters.

Notice here we are not using the IF to subset the records in WDGOT_D, but to assign a value to a variable.

```
PROC SORT DATA = WD_GOT;
  BY STUDYID;
RUN;

PROC SORT DATA = ACTORS;
  BY STUDYID;
RUN;

DATA WDGOT_D;
  MERGE WD_GOT (IN=CHAR) ACTORS (IN=ACT);
  BY STUDYID;

  IF ACT AND ^CHAR THEN SHOW = "EMPIRE";

RUN;

PROC PRINT DATA= WDGOT_D NOOBS; RUN;
```

We will come back to the WDGOT_D dataset later in the lab.

3.3 - Working with Subsets of SAS Datasets

Occasionally you may want to work with only part of the data in a SAS dataset. For example, you might want a small manageable dataset with which to test a new program to see if it performs the way that you expect. You can use the existing SAS dataset as input and create a new dataset that is a subset of the original dataset. Subsets can be created in a variety of ways depending on what you are trying to do.

Subsets based on number of observations

Suppose you want to use only the first 10 observations for your analysis. You can create a new SAS dataset from our ACTORS dataset which includes only the first 10 observations. The new dataset will be called ONE.

```
DATA ONE;  
    SET ACTORS (OBS=10);  
RUN;  
  
PROC PRINT DATA = ONE;  
    TITLE 'CREATING A NEW SAS DATA SET FROM AN ORIGINAL SAS DATA SET';  
RUN;
```

Notice the TITLE line in the print statement. This is a useful way to notate your SAS output in a way that makes it easier to understand what the output represents. The same title will appear in all subsequent output until you provide either a new title or until you turn the title off. This is done with a title statement without an argument as shown below.

```
PROC PRINT DATA = ONE;  
    TITLE;  
RUN;
```

Subsets based on variables

SAS allows you to create a subset of a larger dataset not only by selecting certain observations but also by specifying which variables you want the new dataset to contain. This is accomplished in a DATA step using the KEEP or DROP options. Suppose you want to make a version of the WALKDEAD dataset that contains only the variables for studyid, sex, and numepis. Use the KEEP option to keep those variables only.

```
DATA TWO (KEEP = STUDYID SEX NUMEPIS);  
    SET WALKDEAD;  
RUN;
```

```
PROC PRINT DATA = TWO;  
RUN;
```

In the previous example, it was most efficient to use a KEEP option since there were fewer variables to keep than there were to drop. If on the other hand, you want your dataset to contain most of the existing variables and you want to omit just a few variables, it is more efficient to use the DROP option. The following code could be used to create a new version of the WALKDEAD dataset that contains all variables except for show.

```
DATA TWO (DROP = SHOW);  
    SET WALKDEAD;  
RUN;
```



```
PROC PRINT DATA = TWO;
RUN;
```

**Notice that we called this dataset "TWO", the same name as the previous dataset. SAS wrote over the previous dataset named "Two". Be careful when naming datasets that you don't write over a dataset that you want to save.*

You can use a single DATA step to create more than one dataset at a time. The datasets can be derived from the same original dataset and can contain different contents by using the drop or keep options. Here we will create two datasets from WALKDEAD. The new datasets will be called "TWO" and "THREE."

```
DATA TWO    (KEEP = STUDYID FNAME LNAME)
      THREE (KEEP = STUDYID SEX NUMEPIS SHOW);
SET WALKDEAD;
RUN;
```

```
PROC PRINT DATA = TWO;
      TITLE NAME;
RUN;
```

```
PROC PRINT DATA = THREE;
      TITLE OTHER INFO;
RUN;
```

Notice that there is only one semicolon in the data statement. It is one DATA statement that creates two (or more) new SAS datasets.

Subsets based on values of variables

You may wish to create a subset of a dataset that includes only those subjects who have certain characteristics; that is, subjects who have certain values for one or more of the variables. This is accomplished by using an IF statement in the data step. For example, in the WALKDEAD dataset there is a variable called SEX. This variable is coded 1 if the character is male and 2 if the character is female. Let's first take a look at the distribution of the data in the SEX variable in the dataset.

```
PROC FREQ DATA=WALKDEAD;
      TABLES SEX;
RUN;
```

The following code will create a new temporary dataset that contains only male subjects.

```
DATA FOUR;
      SET WALKDEAD;
      IF SEX=1;
RUN;
```

You can use combinations of variables to populate your database in any way that you want. For example, the following code will create a dataset from WALKDEAD that includes only characters who are male and who starred in less than 60 episodes of the show.

```
DATA FIVE;
    SET WALKDEAD;
    IF SEX=1 AND NUMEPIS < 60;
RUN;
```

You can also use an IF statement to exclude subjects from a new dataset based upon values of one or more variables. For example, the following code creates a new temporary dataset that includes all characters EXCEPT for those who starred in more than 100 episodes.

```
DATA SIX;
    SET WALKDEAD;
    IF NUMEPIS > 100 THEN DELETE;
RUN;
```

Another way to subset a dataset based on values of variables is to use a WHERE statement. However, the WHERE statement is generally used to subset a dataset as part of a procedure (PROC) rather than as part of a DATA step. For example, the following code would be used to take a look at the last names of characters that are female:

```
PROC FREQ DATA=WALKDEAD;
    TABLES LNAME;
    WHERE SEX=2;
RUN;
```

Notice that there was no new dataset created by this program. The WHERE statement allowed us to examine the distribution of one variable (LNAME) conditioned on the value of another variable (SEX).

3.4 - Data Cleaning in SAS

We are now going to do a bit of cleaning and refining. Let's switch over to using the mod3lab SAS dataset you downloaded. First, take a look at the values in all of the variables in our dataset.

```
libname NICOLE 'H:\AEPI537D';

PROC FREQ DATA=NICOLE.mod3lab;
RUN;
```

Notice that the dataset mod3lab has 7 people with daily calorie intake levels of less than 1000. In order to determine if these values make sense, it may be useful to take a look at the other data for these individuals. We can do this easily by sub-setting the data on this variable using a where statement as follows.

```
PROC FREQ DATA=NICOLE.mod3lab;
    TABLES CALORIE;
```

AEPI537D

```

RUN;

PROC PRINT DATA=NICOLE.mod3lab;
  WHERE 0 < CALORIE < 1000;
RUN;

```

The SAS output from this program is shown below:

Obs	studyid	sex	bd	marital	birthoz	weight	heightft	heightin	ill	birthlb	surveydt	kids	house	wtype	smoke	sleep	plane	calorie	fruit	vitamin
25	27	female	6087	single	2	140	5	6	0	9	01/27/1999	0	2	over	0	6	1	800	15	1
207	240	female	7513	single	0	170	5	7	0	6	09/21/2001	0	2	over	0	6	1	800	4	1
216	250	female	6972	single	3	105	5	9	0	7	09/21/2001	0	1	ideal	0	10	1	984	2	0
228	268	female	6657		8	115	5	8	1	6	09/21/2001	0	3	ideal	0	8	.	1	2500	5
242	286	FEMALE	6322	SINGLE	2	135	.	5	0	7	09/21/2001	0	1	1	.	0	6	1	1500	8
249	306	female	3936	single	3	125	5	5	0	9	09/19/2001	1	5	ideal	0	8	1	100	7	1
286	347	FEMALE	6688	MARRIED	10	120	5	4	0	5	09/10/2002	0	2	IDEAL	0	8	0	100	5	0

The first three observations appear to be possible. The next two (studyid 268 and 286) appear to have a problem. It looks like some of the data from their questionnaires were entered into the wrong variable. We will assume that you have checked the original data forms for these people and confirmed that this was the case. The values of 100 calories for the last two people are clearly wrong and since we can't find their true values we will make these values missing. The following code will correct these data entry errors.

```

DATA TEMP;
  SET NICOLE.mod3lab;
  IF STUDYID = 268 THEN PLANE=1;
  IF STUDYID = 268 THEN CALORIE=2500;
  IF STUDYID = 268 THEN FRUIT=5;
  IF STUDYID = 268 THEN VITAMIN=.;
  IF STUDYID = 286 THEN DO; *This illustrates use of a do-loop;
    WTYPE = ' '; *Remember that this is a character variable;
    SMOKE = 0;
    SLEEP = 6;
    PLANE = 1;
    CALORIE = 1500;
    FRUIT = 8;
    VITAMIN = .;
  END;
  IF STUDYID=306 OR STUDYID=347 THEN CALORIE=.;
RUN;

```

Let's check to see if our program worked the way that we intended it to.

```

PROC PRINT;
  WHERE STUDYID IN (268, 286, 306, 347);
RUN;

```

You will be asked to correct the remaining problems in the dataset for the portion of this lab that you submit.

3.5 - Working with Character Data in SAS Datasets

Notice that some of the variables in your TEMP data set are character variables. Storing character data rather than numeric data has a number of drawbacks including additional storage space is required, values may be misspelled, and many SAS analytic procedures will not operate on character variables. In order to change a variable from character to numeric, you must create a new variable in a DATA step. To perform this transformation one typically uses IF/THEN statements. We would like to change the character variables SEX and MARITAL to numeric variables. First run a frequency count of these variables;

```
PROC FREQ DATA=TEMP;
    TABLES SEX MARITAL;
RUN;
```

Notice that the data entry was not consistent for the categories. For example, female sex is coded as 'female' and as 'FEMALE'. Because character variables are case sensitive in SAS, they are considered to be different categories. You must take all variations into account when recoding. We will create new numeric versions of these variables in a new temporary data set called TEMP2.

```
DATA TEMP2;
    SET TEMP;
    IF SEX = 'female' OR SEX = 'FEMALE' THEN SEX1 = 1;
    IF SEX = 'male' OR SEX = 'MALE' THEN SEX1 = 0;
    IF MARITAL = 'single' OR MARITAL = 'SINGLE' OR MARITAL = 'Single' THEN
        MARITAL1 = 1;
    IF MARITAL = 'married' OR MARITAL = 'MARRIED' THEN MARITAL1 = 0;

PROC FREQ DATA = TEMP2;
    TABLES SEX SEX1 MARITAL MARITAL1;
RUN;
```

Again, notice that the character variables had to be specified exactly as they are in the data set (i.e. as they appeared in the frequency tables). Note that when there is more than one condition an OR is needed in the IF/THEN statement. To determine if the new variables are correct we can use PROC FREQ to create **cross-tables** for the old and new variables. To create cross tables use a TABLES statement and the variable names separated by an asterisk;

```
PROC FREQ DATA = TEMP2;
    TABLES SEX*SEX1 MARITAL*MARITAL1;
    TITLE CROSSTABS OF OLD AND NEW VARIABLES;
RUN;
```

The variables for smoking (smoke), plane travel (plane), vitamin use (vitamin), and respiratory illness (ILL) are YES/NO variables and are already coded as 1 for Yes and 0 for No. They are ready for analysis. Numeric variables that store data as numbers that represent levels are said to contain discrete numeric data.

The variables number of children (kids), number in household (house), hours of sleep (sleep), birth weight (birthlb & birthoz), current weight (weight), height (heightft & heightin), caloric intake (calorie), and fruit intake (fruit) contain continuous numeric data. You have already learned how to use SAS procedures such as UNIVARIATE to describe the measures of central tendency and dispersion in your continuous data.

3.6 – Some tips for using IF vs WHERE:

- The only way to limit observations in a PROC step is with a WHERE. IF statements will not work in PROC steps even though WHERE statements work in DATA steps. (Best practice: Always use WHERE in PROC steps.)
- Both IF and WHERE will work in a DATA step, but when you are using WHERE there must be a SET statement since WHERE can only be used to select observations from an already existing data set. Any variable used in a WHERE must exist in that data set. Variables created in a DATA step can't be used in a WHERE in the same DATA step. (Best practice: Always use IF in DATA steps.)
- Note: IF statements are forgiving when you forget how to use numeric and character variables in expressions, but WHERE statements are not. For example, if you reference the value of a numeric variable by putting it in quotes in an IF statement, SAS will convert it to numeric for you and process the IF statement anyway. If you do this in a WHERE statement, SAS will produce an error about incompatible variables in the log and stop processing the step.

MOD 3: USE SAS COMMENTS TO NUMBER YOUR WORK.

1. Create a new dataset called WDGOT_E from the WDGOT_D we created earlier, and fill in some addition information. All of this can be completed using a single data step.

- a. Using DO LOOPS and the STUDYID variable, fill in the following character names (FNAME, LNAME) for the 5 Empire actors using each STUDYID.

STUDYID	FNAME	LNAME
8001	HAKEEM	LYON
8002	JAMAL	LYON
8003	LUCIOUS	LYON
8004	COOKIE	LYON
8005	BECKY	WILLIAMS

- b. Using an IF/THEN statement and the STUDYID variable, assign the SEX for STUDYID 8001, 8002, and 8003 as male (SEX=1).
- c. Using an IF/THEN statement and the STUDYID variable, assign the SEX for STUDYID 8004 and 8005 as female (SEX=2).
- d. Using IF/THEN statements and the ACTOR variable, assign NUMEPIS values for each actor using the following information.

ACTOR	NUMEPIS
BRYSHERE GRAY	49
JUSSIE SMOLLETT	49
TERRENCE HOWARD	49
TARAJI P HENSON	49
GABOUREY SIDIBE	45

- e. Use a print procedure to look at the WDGOT_E dataset.

2. In a temporary SAS dataset called TEMPF, make the following corrections to the data in your TEMP2 dataset:

- a. Change the incorrect weight of 43 lbs to the correct value of 143 lbs.
- b. Set any heightft values that are under 4 feet or over 6 feet to missing.
- c. For the person who has 16 ounces for birthoz, change the value of birthoz to zero and increment the birthlb value by 1 pound (add 1 pound). Set any values of birthoz over 16 ounces to missing.
- d. Set the value of house (persons living in the household) for anything over 7 to missing.
- e. Create a new variable called WTTYPE1 that has the value 0 if the subject considered himself/herself to be at ideal body weight, 1 if underweight and 2 if overweight (based on values in the variable WTTYPE).

3. Create a temporary SAS dataset called TEMPG that contains only the first 100 observations from the TEMPF dataset.
4. Create a temporary dataset called TEMPH that contains data on all variables except SLEEP, PLANE, and FRUIT from the TEMPF dataset.
5. Create a temporary dataset called TEMPI from TEMPF that contains data only for those subjects who did not travel in an airplane in the semester prior to completing the survey (PLANE=0).